

15. Proxy Design Pattern in Java

15.1 Introduction

The **Proxy Design Pattern** is a **structural pattern** that provides a surrogate or placeholder for another object to control access to it.

15.2 Problem it Solves

When you want to add control, delay, or security over an object's access or instantiation without changing the original object code.

15.3 What is Proxy Pattern?

A proxy acts as a stand-in for the real object and can perform additional tasks like access control, logging, or lazy initialization before delegating the call.

15.4 Real-World Analogy

A credit card is a proxy for your bank account. It allows you to interact with your money without directly accessing your bank account.

15.5 Types of Proxy

- **Virtual Proxy:** Delays object creation
- **Protection Proxy:** Controls access based on permissions
- **Remote Proxy:** Access object in another address space (e.g., network)
- **Smart Proxy:** Adds extra behaviour like logging or caching

15.6 Structure

```
package com.mannemlokes;

interface Subject {
    void request();
}

class RealSubject implements Subject {
    public void request() {
        System.out.println("Executing request in RealSubject");
    }
}

public class ProxySubject implements Subject {
    private RealSubject realSubject;

    public void request() {
        if (realSubject == null) {
            realSubject = new RealSubject();
        }
        System.out.println("Proxy: delegating request...");
        realSubject.request();
    }
}
```

15.7 Benefits

- Add additional behaviour without changing actual class
- Useful for lazy loading
- Useful for access control and logging

15.8 Implementation Steps

1. Define a common interface
2. Implement the real object
3. Create a proxy that implements the same interface
4. Add additional behaviour in proxy before/after delegating to the real object

15.9 Examples

Example1: Image Proxy

```
package com.mannemlokesesh;  
  
interface Image {  
    void display();  
}  
  
class RealImage implements Image {  
    private String filename;  
  
    public RealImage(String filename) {  
        this.filename = filename;  
        loadFromDisk();  
    }  
  
    private void loadFromDisk() {  
        System.out.println("Loading: " + filename);  
    }  
  
    public void display() {  
        System.out.println("Displaying: " + filename);  
    }  
}  
  
class ProxyImage implements Image {  
    private RealImage realImage;  
    private String filename;  
  
    public ProxyImage(String filename) {  
        this.filename = filename;  
    }  
  
    public void display() {  
        if (realImage == null) {  
            realImage = new RealImage(filename);  
        }  
        realImage.display();  
    }  
}  
  
public class Example1ProxyImageClient {  
    public static void main(String[] args) {
```

```

        Image image = new ProxyImage("test.jpg");
        image.display();
        image.display();
    }
}

```

Output:

```

Loading: test.jpg
Displaying: test.jpg
Displaying: test.jpg

```

Example2: Internet Access Control

```

package com.mannemlokes;

import java.util.List;

interface Internet {
    void connectTo(String site);
}

class RealInternet implements Internet {
    public void connectTo(String site) {
        System.out.println("Connecting to: " + site);
    }
}

class ProxyInternet implements Internet {
    private RealInternet internet = new RealInternet();
    private static List<String> bannedSites = List.of("abc.com", "xyz.com");

    public void connectTo(String site) {
        if (bannedSites.contains(site)) {
            System.out.println("Access Denied to: " + site);
        } else {
            internet.connectTo(site);
        }
    }
}

public class Example2InternetClient {
    public static void main(String[] args) {
        Internet net = new ProxyInternet();
        net.connectTo("google.com");
        net.connectTo("abc.com");
    }
}

```

Output:

```

Connecting to: google.com
Access Denied to: abc.com

```

Example3: Virtual Proxy for Expensive Objects

```

package com.mannemlokes;

```

```

interface Report {
    void generate();
}

class HeavyReport implements Report {
    public HeavyReport() {
        System.out.println("Initializing heavy report...");
        try {
            Thread.sleep(2000);
        } catch (InterruptedException ignored) {
        }
    }

    public void generate() {
        System.out.println("Generating heavy report...");
    }
}

class ReportProxy implements Report {
    private HeavyReport heavyReport;

    public void generate() {
        if (heavyReport == null) {
            heavyReport = new HeavyReport();
        }
        heavyReport.generate();
    }
}

public class Example3ReportClient {
    public static void main(String[] args) {
        Report report = new ReportProxy();
        System.out.println("Proxy created");
        report.generate();
        report.generate();
    }
}

```

Output:

```

Proxy created
Initializing heavy report...
Generating heavy report...
Generating heavy report...

```

15.10 Proxy vs Other Patterns

Pattern	Purpose
Proxy	Control access to an object
Decorator	Add functionality dynamically
Adapter	Convert interface to another
Facade	Simplify interaction with subsystems

15.11 Pros and Cons

 Pros

- Lazy initialization
- Security and access control
- Logging and performance monitoring

 Cons

- Adds complexity
- Extra layer of indirection may affect performance

15.12 When to Use / Not to Use

Use When:

- Object is expensive to create
- Need controlled access (security)
- Want to add logging/caching without modifying real object

Avoid When:

- You don't need extra control or behaviour
- Simple objects without performance concerns

15.13 Summary

Feature	Description
Intent	Provide controlled access to objects
Use Case	Logging, security, lazy init, caching
Key Benefit	Add behaviour without changing original

The **Proxy Pattern** is an essential tool to wrap and control access to real objects while adding optional logic around them.